

PDP 学业发展规划系统 — 技术说明文档（桌面端版）

1. 项目概述

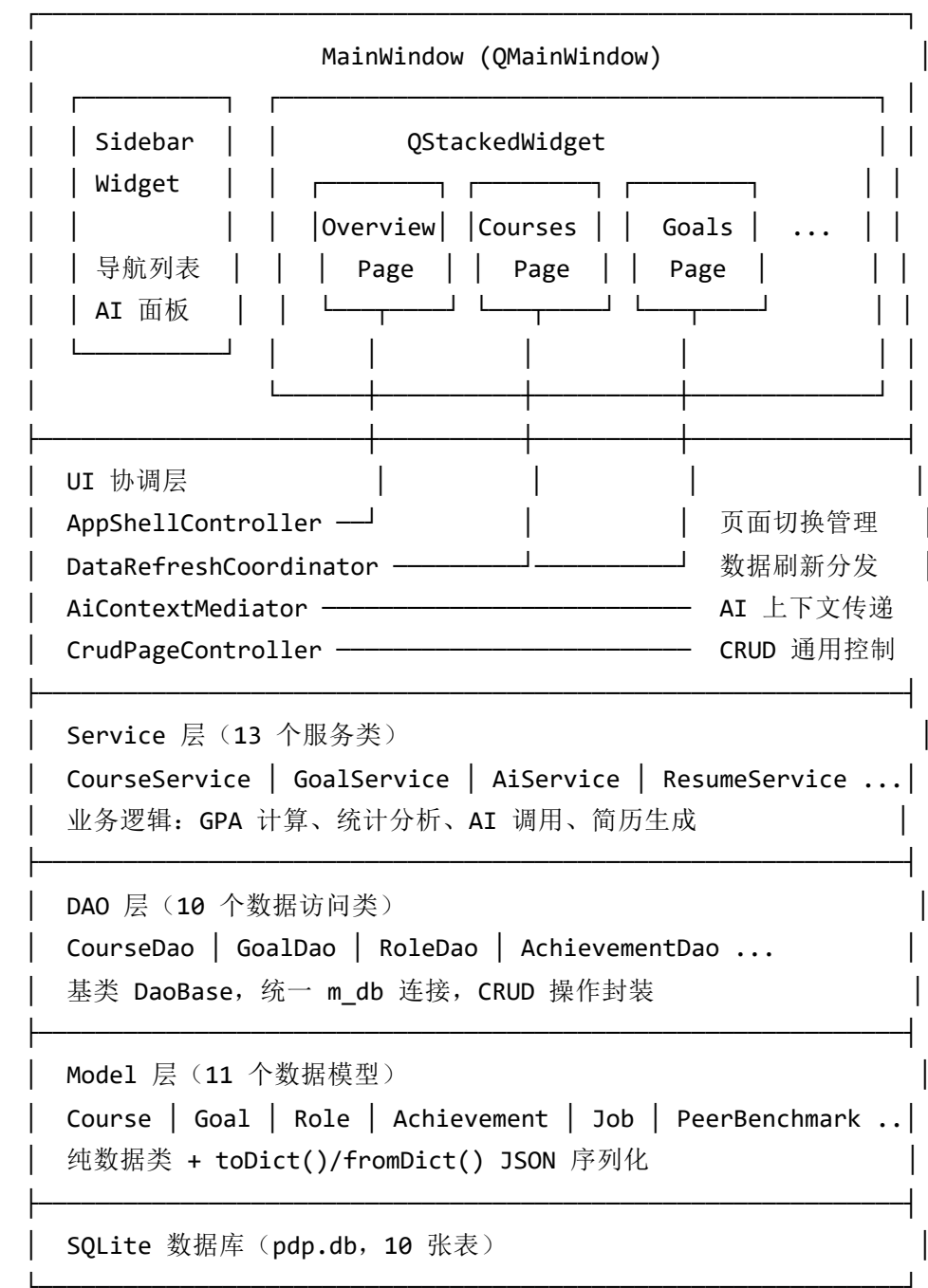
1.1 项目简介

PDP（Personal Development Planning）学业发展规划系统是一款基于 **Qt 6 + C++17** 的桌面端应用程序，帮助学生系统化管理学业生涯中的课程成绩、目标规划、实践经历、课外活动、成就荣誉、岗位追踪等核心数据，并通过 AI 智能分析提供个性化发展建议。

1.2 核心功能

序号	功能模块	说明
1	课程管理	课程信息录入、成绩管理、GPA 自动计算（加权/标准两种算法）
2	目标规划	学期目标设定、里程碑追踪、进度百分比自动计算
3	角色任职	学生干部/社团角色管理、任期追踪
4	成就荣誉	奖学金、竞赛获奖等成就记录
5	实践经历	实习、志愿服务等实践经历管理
6	课外活动	社团活动、讲座等课外活动记录
7	岗位追踪	求职目标岗位管理、技能需求对照、完成度追踪
8	数据分析	学期对比、同学对照、雷达图、GPA 趋势图
9	AI 智能分析	基于全部数据的 AI 综合分析与建议
10	简历生成	基于个人数据自动生成 JSON/HTML 格式简历

1.3 系统架构



四层架构: UI (Pages/Dialogs/Widgets + 4 个 UI 协调器) → Service (业务逻辑) → DAO (数据访问) → Model (数据模型 + SQLite)

1.4 技术栈

技术	版本	用途
C++	C++17	主开发语言

技术	版本	用途
Qt	6.5+	GUI 框架（Qt Widgets）
SQLite3	内置	本地数据存储（QSqlDatabase / QSqlQuery）
CMake	3.25+	构建系统
QNetworkAccessManager	Qt 6.x	HTTP 网络请求（AI 服务调用）

2. 模块详解

2.1 Model 层（11 个模型类）

所有模型定义在 `src/model/` 目录下，均为纯数据类，提供 `toDict()` / `fromDict()` 进行 JSON 序列化。

模型类	说明	特殊方法
Course	课程模型	<code>calculateGradePoint()</code> 绩点计算
Role	角色任职	—
Achievement	成就荣誉	—
Experience	实践经历	—
Activity	课外活动	—
Goal	目标规划	<code>progress()</code> 进度百分比
Job	目标岗位	—
JobRequirement	岗位技能需求	—
PeerBenchmark	同学对照	GPA 计算
ImportResult	导入结果	—
ImportErrorItem	导入错误条目	—

示例代码（Course 模型）：

```
class Course {
public:
    int id = 0;
    QString name;           // 课程名称
    QString code;           // 课程代码
    double credits = 0;     // 学分
    QString semester;       // 学期
    QString category;       // 类别
    double score = 0;       // 成绩
    double gradePoint = 0;  // 绩点
    QString status;         // 状态
    QString teacher;        // 教师
    QString location;        // 地点
    QString description;    // 描述
    QStringList tags;       // 标签

    QJsonObject toDict() const;           // 序列化为 JSON
    static Course fromDict(const QJsonObject& obj); // 从 JSON 反序列化
    static double calculateGradePoint(double score, const QString& category); // 绩点计算
};
```

2.2 DAO 层（10 个数据访问类）

所有 DAO 定义在 src/dao/ 目录下，继承自 DaoBase 基类。

DaoBase 基类：

- 持有 QSqlDatabase m_db 数据库连接
- 提供 isOpen() 检查连接状态
- 统一的 CRUD 操作封装

DAO 类	对应表	说明
CourseDao	courses	课程增删改查
RoleDao	roles	角色任职
AchievementDao	achievements	成就荣誉
ExperienceDao	experiences	实践经历
ActivityDao	activities	课外活动

DAO 类	对应表	说明
GoalDao	goals	目标规划
JobDao	jobs	岗位追踪
PeerBenchmarkDao	peer_benchmarks	同学对照

示例代码（CourseDao::create）：

```
bool CourseDao::create(Course& course) {
    QSqlQuery query(m_db);
    query.prepare(
        "INSERT INTO courses (name, code, credits, semester, category, score, "
        "grade_point, status, teacher, location, description, tags) "
        "VALUES (:name, :code, :credits, :semester, :category, :score, "
        ":grade_point, :status, :teacher, :location, :description, :tags)");
    query.bindValue(":name", course.name);
    query.bindValue(":code", course.code);
    query.bindValue(":credits", course.credits);
    query.bindValue(":score", course.score > 0 ? QVariant(course.score) : QVariant());
    query.bindValue(":grade_point", course.gradePoint > 0 ? QVariant(course.gradePoint) : QVariant());
    // ... 其余字段绑定
    if (!query.exec()) return false;
    course.id = query.lastInsertId().toInt();
    return true;
}
```

2.3 Service 层（13 个服务类）

所有 Service 定义在 src/service/ 目录下，是业务逻辑的核心层。除 AiService 外均为 Header-Only（仅 .h 文件）。

Service 类	职责
CourseService	课程管理 + GPA 计算
RoleService	角色任职管理
AchievementService	成就荣誉管理
ExperienceService	实践经历管理

Service 类	职责
ActivityService	课外活动管理
GoalService	目标规划管理
JobService	岗位追踪管理
AiService	AI 分析（唯一有 .cpp 的 Service）
AnalyticsService	数据分析（含 PeerBenchmarkService）
DashboardService	仪表盘数据汇总
ImportService	CSV 数据导入
ResumeService	简历生成（JSON/HTML）

AiService 核心流程：

```
analyze()  
├─ 检查缓存（24h 有效期内直接返回）  
├─ 重置服务器状态 → resetAiServerCheck()  
├─ 探测本地 AI（GET /health, QNetworkAccessManager）  
│   ├─ 成功 → analyzeWithAi() → callAiServer()  
│   └─ 失败 → 探测远程 AI（GET /health）  
│       └─ 成功 → analyzeWithAi()  
│       └─ 失败 → analyzeLocal() 本地模板分析  
└─ 缓存结果
```

2.4 UI 协调层（4 个核心协调器）

定义在 src/client/core/ 目录下，负责 UI 层的协调和通信。

协调器	基类	职责
AppShellController	QObject	管理 QStackedWidget 页面切换、侧边栏联动
DataRefreshCoordinator	QObject	监听 dataChanged(DataDomain) 信号， 分发刷新到对应页面
AiContextMediator	QObject	通过 eventFilter() 拦截全局鼠标事件， 将选中文本推送到 AI 面板

协调器	基类	职责
CrudPageController	—	模板控制器，用 <code>std::function</code> 注册表替代 switch-case

DataRefreshCoordinator 工作机制：

- 数据变更 → 发出 `dataChanged(DataDomain)` 信号
- `DataRefreshCoordinator` 接收信号
 - `refreshByDomain(domain)` 分发到对应页面
 - 页面.`refresh()` 重新加载数据

AiContextMediator 的 eventFilter 实现：

```
bool AiContextMediator::eventFilter(QObject* watched, QEvent* event) {  
    // 拦截鼠标事件，检测文本选中  
    // 将选中文本推送到 AI 面板  
    // 实现"任意位置选中文字 → AI 分析"的全局交互  
}
```

2.5 Page 层（13 个页面）

定义在 `src/client/pages/` 目录下，均继承自 `BasePage` 基类。

BasePage 基类：

```
class BasePage : public QWidget {  
    Q_OBJECT  
public:  
    explicit BasePage(QWidget* parent = nullptr);  
    ~BasePage() override;  
    virtual void refresh() = 0; // 纯虚函数，子类必须实现  
protected:  
    QFrame* createMetricCard(const QString& labelText, QLabel** valueLabel,  
                             const QString& helperText = QString());  
};
```

页面类	功能
OverviewPage	总览仪表盘
CoursesPage	课程管理

页面类	功能
RolesPage	角色任职
AchievementsPage	成就荣誉
ExperiencesPage	实践经历
ActivitiesPage	课外活动
GoalsPage	目标规划
JobsPage	岗位追踪
AnalysisPage	数据分析
TimelinePage	时间线
ResumePage	简历生成
ImportsPage	数据导入

2.6 Dialog 层（9 个编辑对话框）

定义在 `src/client/dialogs/` 目录下，均为模态对话框，用于数据的新增和编辑。

对话框类	功能
ProfileEditorDialog	个人信息编辑
CourseEditorDialog	课程编辑
RoleEditorDialog	角色编辑
AchievementEditorDialog	成就编辑
ExperienceEditorDialog	经历编辑
ActivityEditorDialog	活动编辑
GoalEditorDialog	目标编辑
JobEditorDialog	岗位编辑
PeerEditorDialog	同学数据编辑

2.7 Widget 层（17 个自定义组件）

定义在 `src/client/widgets/` 目录下，是可复用的 UI 组件。

组件类	功能
SidebarWidget	侧边栏（QFrame，折叠动画 QPropertyAnimation）
NavigationListWidget	导航列表（QListWidget）
AiPanelWidget	AI 面板（状态指示 + 对话 + 建议）
AiConversationWidget	AI 对话（流式消息渲染）
AiStatusBar	AI 状态栏（实时状态指示）
ToastNotification	消息提示（QPropertyAnimation 滑入滑出）
TimeInfoCard	时间信息卡片
StudentInfoCard	学生信息卡片（QSettings 读写）
MetricGridWidget	指标网格
SuggestionListWidget	AI 建议列表
CrudPageShell	CRUD 页面外壳
SemesterAnalysisWidget	学期分析（双学期 GPA 对比）
PeerBenchmarkWidget	同学对照（曼哈顿距离 + 雷达图）
ResumePreviewWidget	简历预览
ResumePreviewDialog	简历预览对话框
ResumeEditorPanel	简历编辑面板
ResumeCandidatePanel	简历候选面板

2.8 工具类

类	文件	功能
Logger	src/util/Logger.h	四级日志（info/warning/error/debug）

类	文件	功能
JsonUtils	src/util/JsonUtils.h	JSON 响应构建 (successResponse/errorResponse)
UiHelpers	src/client/utils/UiHelpers.h	UI 辅助工具函数

3. Qt 核心类使用

3.1 布局与容器

Qt 类	用途
QMainWindow	MainWindow 主窗口基类
QWidget	所有 Page、Widget 的基类
QFrame	卡片容器、分割线
QDialog	所有编辑对话框的基类
QVBoxLayout / QHBoxLayout / QGridLayout / QFormLayout	布局管理
QSplitter	可拖拽分割（简历编辑器左右分栏）
QScrollArea	可滚动区域（Dashboard、详情页）
QStackedWidget	页面切换容器（MainWindow 核心）

3.2 数据与存储

Qt 类	用途
QSqlDatabase	SQLite 数据库连接（QSqlDatabase::database()）
QSqlQuery	SQL 执行（prepare() + bindValue() 参数化查询）
QJsonObject	Model 的 toDict() / fromDict() 序列化
QJsonArray	JSON 数组（tags、milestones）
QJsonDocument	JSON 文档解析

Qt 类	用途
QSettings	持久化配置（学生姓名、头像路径、窗口状态）

3.3 网络

Qt 类	用途
QNetworkAccessManager	HTTP 请求（AiService 调用 AI 服务）

3.4 UI 组件

Qt 类	用途
QTableWidget	课程、岗位等数据表格
QSystemTrayIcon	系统托盘图标
QProgressBar	进度条（目标进度、加载状态）
QLabel	文本显示
QToolBar	工具栏
QGraphicsOpacityEffect	透明度效果（Toast 动画）
QPropertyAnimation	属性动画（侧边栏折叠、Toast 滑入滑出）

3.5 C++ 标准库特性

特性	用途
std::function	CrudPageController 的注册表（CreateDialogFn、RefreshFn）
std::sort + lambda	GPA 排序
std::min	取最小值
std::move	JsonUtils::applyCommonHeaders() 中的移动语义

4. 关键技术实现

4.1 QTableWidgetItem 表格管理

以 CoursesPage 为例，展示 Qt 表格的标准用法：

```
QTableWidgetItem* table = new QTableWidgetItem(this);
table->setColumnCount(7);
table->setHorizontalHeaderLabels({"课程名称", "代码", "学分", "学期", "成绩", "绩点", "状态"});
table->setSelectionBehavior(QAbstractItemView::SelectRows);
table->setSelectionMode(QAbstractItemView::SingleSelection);
table->setEditTriggers(QAbstractItemView::NoEditTriggers);
table->horizontalHeader()->setStretchLastSection(true);
table->setSortingEnabled(true);
```

数据填充由 Service 层提供，通过 CourseService::getAll() 获取 QList<Course>，遍历后填入表格行。setSortingEnabled(true) 支持点击表头排序。

4.2 QSqlQuery 参数化查询

使用 prepare() + bindValue() 防止 SQL 注入：

```
QSqlQuery query(m_db);
query.prepare("INSERT INTO courses (name, code, credits, ...) VALUES (:name, :code, :credits, ..");
query.bindValue(":name", course.name);
query.bindValue(":code", course.code);
query.bindValue(":score", course.score > 0 ? QVariant(course.score) : QVariant());
query.exec();
```

注意 score 和 grade_point 字段使用 QVariant() 处理空值（映射到 SQL NULL）。

4.3 CrudPageController 策略模式

用 std::function 注册表替代传统的 switch-case：

```
using CreateDialogFn = std::function<QJsonObject(QWidget*)>;
using RefreshFn = std::function<void()>;

// MainWindow 中的页面刷新注册表
QMap<int, std::function<void()>> m_pageRefreshers;
```

每种页面类型注册自己的创建对话框函数和刷新函数，新增页面类型只需注册，无需修改控制器代码。

4.4 DataRefreshCoordinator 星型拓扑

12 个页面不直接互相通信，而是通过协调器中转：

```
// 页面发出信号
emit dataChanged(DataDomain::Courses);

// 协调器接收并分发
void DataRefreshCoordinator::refreshByDomain(DataDomain domain) {
    switch (domain) {
        case DataDomain::Courses:
            m_overviewPage->refresh();
            m_analysisPage->refresh();
            break;
        // ...
    }
}
```

4.5 AiContextMediator 全局事件拦截

通过 Qt 的 `eventFilter()` 机制实现"任意位置选中文字 → AI 分析"：

```
bool AiContextMediator::eventFilter(QObject* watched, QEvent* event) {
    if (event->type() == QEvent::MouseButtonRelease) {
        // 检测文本选中
        // 将选中文本推送到 AI 面板
    }
    return QObject::eventFilter(watched, event);
}
```

所有 Widget 安装此事件过滤器后，用户在任何页面选中文字都会自动触发 AI 分析。

4.6 AiService 双模式 AI

支持本地 AI 和远程 AI 两种模式，自动降级：

```
// 探测本地 AI (localhost:5001/health)
// 探测远程 AI (公网地址/health)
// 本地优先 → 远程降级 → 本地模板兜底
```

4.7 侧边栏折叠动画

```
QPropertyAnimation* animation = new QPropertyAnimation(m_sidebar, "maximumWidth");
animation->setDuration(250);
animation->setEasingCurve(QEasingCurve::InOutCubic);
animation->setStartValue(200);
animation->setEndValue(60);
animation->start(QAbstractAnimation::DeleteWhenStopped);
```

4.8 Toast 消息通知

```
// 使用 QGraphicsOpacityEffect + QPropertyAnimation
QGraphicsOpacityEffect* opacityEffect = new QGraphicsOpacityEffect(toast);
toast->setGraphicsEffect(opacityEffect);
QPropertyAnimation* fadeIn = new QPropertyAnimation(opacityEffect, "opacity");
fadeIn->setDuration(300);
fadeIn->setStartValue(0.0);
fadeIn->setEndValue(1.0);
```

5. 数据库设计

5.1 表结构（10 张表）

表名	说明	主要字段
courses	课程	name, code, credits, semester, score, grade_point, status
roles	角色任职	name, organization, start_date, end_date, description

表名	说明	主要字段
achievements	成就荣誉	title, category, date, description, issuer
experiences	实践经历	title, company, start_date, end_date, description
activities	课外活动	name, category, date, location, description
goals	目标规划	title, description, deadline, progress, status, milestones
jobs	岗位追踪	title, company, requirements, status
job_requirements	岗位技能需求	job_id, name, completed
peer_benchmarks	同学对照	name, gpa, scores (JSON)
timeline	时间线	事件记录

5.2 外键约束

schema.sql 中启用了 PRAGMA foreign_keys = ON ，确保 job_requirements 表的 job_id 外键约束生效。

6. 项目目录结构

```
Qt_Cpp_Final_Submission/
├─ 01_SourceCode/
│   ├── CMakeLists.txt
│   ├── src/
│   │   ├── model/           # 11 个数据模型
│   │   ├── dao/             # 10 个 DAO (含 DaoBase 基类)
│   │   ├── service/         # 13 个服务类
│   │   ├── client/
│   │   │   ├── core/        # 4 个 UI 协调器
│   │   │   ├── pages/       # 13 个页面
│   │   │   ├── dialogs/     # 9 个编辑对话框
│   │   │   ├── widgets/     # 17 个自定义组件
│   │   │   ├── mainwindow/  # MainWindow
│   │   │   └─ utils/        # UI 辅助工具
│   │   └─ util/             # Logger, JsonUtils
│   ├── resources/
│   │   ├── schema.sql       # 数据库建表脚本
│   │   ├── resources.qrc    # Qt 资源文件
│   │   └─ icons/            # 图标资源
│   ├── main_desktop.cpp     # 桌面端入口
│   └─ pdp.db                # SQLite 数据库
├─ 02_Executable/           # 预构建可执行文件
├─ 03_Reports/               # 设计报告
├─ 04_Presentation/         # 答辩材料
└─ 05_RunGuide/             # 运行指南
```

7. 代码统计

指标	数值
桌面端源文件数	137 个 (.h + .cpp)
桌面端代码行数	~15,500 行
Model 类	11 个
DAO 类	10 个

指标	数值
Service 类	13 个
UI 协调器	4 个
Page 类	13 个
Dialog 类	9 个
Widget 类	17 个
数据库表	10 张

(文档结束)